

PROGRAMME DE FORMATION · VERSION 3 · 2026-09-13

Python, programmation objet

Passez de scripts Python à une petite application structurée en objets. Vous modélisez le domaine, choisissez entre composition et héritage, protégez les échanges et testez une interface simple sans coupler toute la logique à l'affichage.

3 jours · 21 heures

Intermédiaire · Hybride

2 440 € HT

Tarif indicatif, dates, disponibilité et modalités contractuelles à confirmer avec le centre.

À qui s'adresse cette formation ?

Développeurs et professionnels disposant de premières bases Python, souhaitant structurer une application en objets.

Prérequis

Savoir utiliser variables, conditions, boucles et fonctions Python. Lire et modifier un script simple ; la conception objet est introduite pendant le parcours.

Objectifs pédagogiques

- Comprendre les bases du langage Python et son écosystème
- Acquérir les principes de la programmation objet
- Comprendre et utiliser les fonctions et modules
- Concevoir des interfaces graphiques
- Utiliser les outils de test et d'évaluation de la qualité d'un programme Python

Le cas fil rouge

Structurer un gestionnaire d'inventaire Python avec modèle objet, interface simple et tests automatisés.

Préparation

Premières fonctions et collections Python ; Python, éditeur, pytest et Tkinter disponibles dans le poste de formation.

Progression détaillée

Répartition pédagogique proposée sur une base de 7 heures par jour, pauses exclues. À ajuster avec le formateur au positionnement initial. Les approfondissements « Pour aller plus loin » sont sélectionnés par le formateur selon le positionnement et le temps disponible ; ils ne constituent pas des heures supplémentaires garanties.

Jour 1 · 7 heures

1. Consolider fonctions et modules

210 minutes · apports, exercice et retour

- Organiser fichiers, imports et environnement virtuel.

- Préciser arguments, valeurs de retour et exceptions.
- Distinguer état local, état partagé et effets de bord.
- Clarifier portée, mutabilité, arguments par défaut et annotations ; structurer un paquet sans imports circulaires.

NOTIONS ET MÉTHODES EN DÉTAIL

- Distinguer argument mutable, copie et référence ; éviter une valeur par défaut partagée.
- Organiser modules et imports pour réduire dépendances circulaires et effets au chargement.
- Employer annotations de types, exceptions ciblées et documentation du contrat des fonctions.

Atelier prévu

Transformer un script monolithique d'inventaire en trois modules sans modifier ses résultats.

Livrable attendu

Paquet Python organisé et fonctions aux contrats explicites.

Vérification en atelier

L'import d'un module ne lance pas l'application et les erreurs attendues sont testées.

Pour aller plus loin · selon positionnement et temps disponible

Remplacer des variables globales par des dépendances explicites pour rendre le traitement testable.

2. Modéliser avec des classes

210 minutes · apports, exercice et retour

- Définir attributs, initialisation, méthodes et invariants.
- Utiliser propriétés et dataclasses quand elles simplifient le modèle.
- Distinguer identité, égalité et représentation d'un objet.
- Définir invariants, attributs, propriétés et représentation des objets ; comparer classe classique et dataclass.

NOTIONS ET MÉTHODES EN DÉTAIL

- Définir état, invariants, constructeur et méthodes ; distinguer attribut de classe et d'instance.
- Comparer classe classique et dataclass selon validation, égalité et mutabilité.
- Encapsuler une modification d'état et choisir une représentation lisible pour le débogage.

Atelier prévu

Créer un article dont le stock reste valide après entrée et sortie ; tester une quantité négative et un stock insuffisant.

Livrable attendu

Modèle objet avec invariants et exemples d'usage.

Vérification en atelier

Un objet ne peut pas entrer dans un état invalide par une opération publique prévue.

Pour aller plus loin · selon positionnement et temps disponible

Concevoir un objet qui refuse un état incohérent et reste compréhensible lors de son affichage ou de sa sérialisation.

Jour 2 · 7 heures

3. Choisir composition et héritage

210 minutes · apports, exercice et retour

- Comparer composition, délégation et spécialisation.
- Définir un contrat de comportement sans couplage aux détails.
- Comprendre polymorphisme et responsabilité unique.
- Distinguer relation est-un et possède-un ; examiner couplage, polymorphisme et substitution dans les contrats.

NOTIONS ET MÉTHODES EN DÉTAIL

- Utiliser composition pour déléguer une responsabilité et héritage pour une substitution justifiée.
- Définir un contrat commun avec protocole ou classe abstraite ; tester les implémentations.
- Examiner MRO et super sur un exemple limité sans construire une hiérarchie artificielle.

Atelier prévu

Ajouter deux politiques de remise sans multiplier les conditions dans le calcul de commande.

Livrable attendu

Diagramme de conception et refactoring argumenté.

Vérification en atelier

Une implémentation est remplacée sans modifier le code consommateur ni casser son contrat.

Pour aller plus loin · selon positionnement et temps disponible

Remplacer une hiérarchie fragile par une composition et vérifier la compatibilité des comportements attendus.

4. Fiabiliser les échanges de données

210 minutes · apports, exercice et retour

- Lire et écrire JSON ou CSV avec gestion des erreurs.
- Encapsuler accès aux fichiers et conversions.
- Employer gestionnaires de contexte et journalisation utile.
- Valider les données aux frontières, gérer exceptions métier et ressources avec des gestionnaires de contexte.

NOTIONS ET MÉTHODES EN DÉTAIL

- Séparer sérialisation et objets métier ; valider champs, types et version d'un document JSON.
- Utiliser gestionnaires de contexte pour fermer fichiers et connexions même en cas d'erreur.
- Traduire une erreur d'entrée en exception métier tout en conservant sa cause technique.

Atelier prévu

Construire un dépôt d'articles et gérer un fichier absent, corrompu ou contenant un article invalide.

Livrable attendu

Couche d'échange robuste et catalogue d'erreurs.

Vérification en atelier

Un fichier incorrect est refusé sans création partielle d'objets incohérents.

Pour aller plus loin · selon positionnement et temps disponible

Simuler un fichier corrompu ou une interruption d'écriture et préserver un état cohérent de l'application.

Jour 3 · 7 heures

5. Créer une interface graphique simple

210 minutes · apports, exercice et retour

- Relier widgets, événements et validation utilisateur.
- Séparer affichage, logique métier et persistance.
- Éviter les traitements bloquants dans la boucle d'événements.
- Séparer widgets, état de l'interface et logique métier ; expliquer la boucle d'événements et les traitements bloquants.

NOTIONS ET MÉTHODES EN DÉTAIL

- Construire fenêtres, widgets et callbacks avec une séparation entre interface et services.
- Valider une saisie, afficher un retour et empêcher une action incompatible avec l'état courant.
- Comprendre la boucle d'événements et repérer un traitement long qui bloque l'interface.

Atelier prévu

Créer une fenêtre de consultation et de saisie d'articles qui réutilise les classes métier testées.

Livrable attendu

Interface événementielle et découpage présentation/métier.

Vérification en atelier

La même opération métier fonctionne depuis l'interface et depuis un test sans fenêtre.

Pour aller plus loin · selon positionnement et temps disponible

Maintenir l'interface réactive pendant une opération longue et éviter les modifications concurrentes de son état.

6. Tester et améliorer la conception

210 minutes · apports, exercice et retour

- Écrire tests unitaires, fixtures et cas limites.
- Utiliser types et analyse statique pour clarifier les contrats.
- Refactorer en conservant les comportements validés.
- Tester comportements publics, cas limites et dépendances externes ; distinguer mock utile et test lié aux détails internes.

NOTIONS ET MÉTHODES EN DÉTAIL

- Tester invariants, erreurs et transitions avec fixtures et dépendances remplacées.
- Repérer responsabilités trop larges et extraire une collaboration sans changer le comportement.
- Comparer duplication, couplage et lisibilité avant/après une refonte ciblée.

Atelier prévu

Livrer le gestionnaire avec tests, notice et schéma des responsabilités ; démontrer l'ajout d'un nouveau stockage sans réécrire l'interface.

Livrable attendu

Suite de tests et dossier de décisions de conception.

Vérification en atelier

Les tests de comportement restent verts après le changement de conception.

Pour aller plus loin · selon positionnement et temps disponible

Faire évoluer une règle métier avec une suite de non-régression puis mesurer la réduction du couplage.

Évaluation finale

Mise en situation individuelle et restitution commentée, incluses dans la dernière séquence. Durée indicative : 60 minutes.

- Les invariants métier sont protégés par le modèle.
- La logique reste testable sans interface graphique.
- Les erreurs de fichiers produisent un comportement explicite.

Méthodes et ressources

Apports illustrés, exercices progressifs, ateliers sur un cas fil rouge et retour argumenté sur les productions. Les ateliers et l'évaluation font partie des heures indiquées.

Les ateliers décrivent les travaux prévus. Leurs fichiers, corrigés et évaluations en ligne sont à préparer, sauf ressources explicitement disponibles dans le module pilote.

Accès, accompagnement et contact

Pour obtenir plus de détails ou le programme détaillé validé, contactez le centre : 09 54 70 23 80 — contact@beyondexpertise.eu.

Positionnement et vérification des prérequis avant inscription. Délais, calendrier et aménagements à convenir avec le centre. Pour un besoin d'accessibilité, décrivez le besoin pratique sans diagnostic médical.

Programme enrichi proposé, à valider par le formateur avant animation. Les approfondissements sont adaptés au niveau et au temps disponible. Aucune certification professionnelle ni prise en charge CPF ou OPCO n'est garantie. Démarche Qualiopi en cours ; certification non acquise.

Références pour approfondir

- [Python · tutoriel du langage](#)